ESTÁNDARES DE DESARROLLO PARA ETL

CONTROL DE CAMBIO AL DOCUMENTO:

FECHA	VERSIÓN	RESPONSABLE	DESCRIPCIÓN DEL CAMBIO
26/02/2024	1.0	Javier Morales	Elaboración del documento de estándar de desarrollo para ETL en Seguros Alfa.

Tabla de Contenido

1. OBJETIVO 4

2. INTRODUCCIÓN 4

3. Convenciones de Nombres (Naming Conventions) 4

3.1. Tablas Temporales y de Staging 4

3.2. Scripts y Procesos ETL 4

3.3. Archivos de Entrada y Salida 4

4. Desarrollo de Procesos ETL 5

4.1. Extracción de Datos 5

4.2. Transformación de Datos 5

4.3. Carga de Datos 5

4.4. Desarrollo de Condicionales y Bucles 5

4.5. Documentación del Código y Scripts 5

5. Optimización y Buenas Prácticas 6

5.1. Mejores Prácticas para la Fase de Extracción 6

5.2. Mejores Prácticas para la Fase de Transformación 6

5.3. Mejores Prácticas para la Fase de Carga 6

6. Monitoreo y Control de Calidad 7

6.1. Registro de Ejecuciones (Logging) 7

6.2. Validación y Control de Calidad 7

7. Gestión de Versionamiento 7

7.1. Uso de Git 7

7.2. Control de Versiones en Datos 7

8. CONCLUSIÓN 8

1. OBJETIVO

Este documento tiene como compromiso establecer un conjunto de estándares y mejores prácticas para el desarrollo de procesos ETL en Seguros Alfa. Su implementación garantiza calidad, seguridad, mantenibilidad y eficiencia en la gestión de datos, asegurando la confiabilidad de la información procesada y facilitando la integración entre sistemas.

2. INTRODUCCIÓN

Este documento establece los estándares y mejores prácticas para el desarrollo de procesos ETL (Extract, Transform, Load) en **Seguros Alfa**. Su objetivo es garantizar la calidad, integridad y eficiencia en la extracción, transformación y carga de datos, asegurando la confiabilidad de la información utilizada en los sistemas de la empresa.

3. CONVENCIONES DE NOMBRES (NAMING CONVENTIONS)

El uso de convenciones de nombres adecuadas mejora la comprensión y mantenimiento de los procesos ETL. En esta sección se definen los lineamientos para la nomenclatura de objetos ETL.

3.1. Tablas Temporales y de Staging

- Usar prefijo TMP_ para tablas temporales.
- Usar prefijo STG_ para tablas de staging.

Ejemplo: TMP_CUSTOMERS, STG_TRANSACTIONS

3.2. Scripts y Procesos ETL

- Usar prefijo ETL_ seguido del nombre descriptivo del proceso.

Ejemplo: ETL_LOAD_POLICY_DATA, ETL_CLEANSE_CUSTOMER_RECORDS

3.3. Archivos de Entrada y Salida

- Usar prefijo IN_ para archivos de entrada y OUT_ para archivos de salida.
- Incluir la fecha en el formato YYYYMMDD para trazabilidad.

Ejemplo: IN_SALES_20240201.csv, OUT_SUMMARY_20240201.txt

4. DESARROLLO DE PROCESOS ETL

Esta sección describe las mejores prácticas para la construcción de procesos ETL eficientes y escalables.

4.1. Extracción de Datos

- Utilizar consultas optimizadas para evitar impacto en las bases de datos origen.
- Evitar el uso de SELECT *; especificar las columnas requeridas.

4.2. Transformación de Datos

- Implementar reglas de validación y limpieza de datos.
- Documentar todas las reglas de negocio aplicadas en la transformación.

4.3. Carga de Datos

- Utilizar operaciones en batch para mejorar el rendimiento.
- Aplicar control de duplicados y validaciones antes de insertar datos.

4.4. Desarrollo de Condicionales y Bucles

- Utilizar estructuras de control como IF-THEN-ELSE de manera eficiente para evitar anidamientos innecesarios.
- Optimizar el uso de estructuras condicionales mediante CASE en SQL y funciones predefinidas en herramientas ETL.
- Implementar bucles (FOR, WHILE, LOOP) únicamente cuando sean necesarios, asegurando condiciones de salida claras para evitar iteraciones infinitas.
- Priorizar operaciones en lote sobre iteraciones fila por fila para mejorar el rendimiento en procesos ETL.

Ejemplo de condicional en SQL:

```
CASE
   WHEN status = 'A' THEN 'Active'
   WHEN status = 'I' THEN 'Inactive'
   ELSE 'Unknown'
END AS status_description;
```

Ejemplo de bucle en Python:

```
for file in file_list:
   process_file(file)
```

4.5. Documentación del Código y Scripts

- Documentar el código y los scripts utilizados en los procesos ETL.
- Incluir comentarios, anotaciones o descripciones con información sobre el propósito, los parámetros y los resultados de funciones, procedimientos o consultas.
- Proporcionar ejemplos o fragmentos de código cuando sea relevante.

Ejemplo:

```
# Esta función extrae datos de un archivo CSV y devuelve un dataframe de pandas
def extract_data(file):
   import pandas as pd
   df = pd.read_csv(file)
```

```
    return df
```

Al documentar el código y los scripts, puede hacer que sus procesos ETL sean más legibles, comprensibles y reutilizables, y facilitar la depuración, las pruebas y el mantenimiento.

5. OPTIMIZACIÓN Y BUENAS PRÁCTICAS

5.1. Mejores Prácticas para la Fase de Extracción

Las mejores prácticas para la fase de extracción de ETL incluyen:

- Minimizar el impacto en los sistemas origen: Programar extracciones en horarios de baja carga y evitar consultas que consuman demasiados recursos.
- Extraer solo los datos necesarios: Reducir el volumen de datos seleccionando solo las columnas y filas requeridas.
- Utilizar técnicas de extracción incremental: Implementar procesos que solo extraigan datos nuevos o modificados en lugar de extraer todo el conjunto de datos.
- Monitorear la calidad de los datos: Validar los datos extraídos para identificar registros erróneos o incompletos antes de la transformación.
- Garantizar la seguridad de los datos: Usar conexiones seguras y cifrar datos sensibles durante la extracción.

5.2. Mejores Prácticas para la Fase de Transformación

Las mejores prácticas para la fase de transformación de ETL incluyen:

- Validar y manejar errores: Implementar controles de validación y mecanismos de manejo de errores.
- Optimizar el rendimiento: Utilizar técnicas como procesamiento paralelo, almacenamiento en caché o procesamiento por lotes.
- Reutilizar y modularizar: Diseñar componentes de transformación reutilizables y modulares.
- Documentar y controlar versiones: Documentar la lógica de transformación y utilizar sistemas de control de versiones.
- Limpiar y normalizar datos: Eliminar registros duplicados, completar valores faltantes y estandarizar formatos de datos.
- Configurar alertas de error: Identificar errores de manera oportuna y prevenir la pérdida de datos.
- Optimizar transformaciones: Evitar transformaciones innecesarias o redundantes.
- Mantener la integridad de los datos: Utilizar técnicas como sumas de comprobación, rutinas de validación de datos y áreas de almacenamiento temporales.
- Optimizar el rendimiento del sistema: Programar cuidadosamente y utilizar técnicas de equilibrio de carga.

5.3. Mejores Prácticas para la Fase de Carga

Las mejores prácticas para la fase de carga de ETL incluyen:

- Realizar cargas en modo batch: Procesar grandes volúmenes de datos en lotes en lugar de transacciones individuales para mejorar el rendimiento.
- Implementar control de duplicados: Evitar la inserción de registros duplicados

mediante claves primarias y restricciones de unicidad.
- Aplicar estrategias de particionamiento: Distribuir los datos en particiones para mejorar la velocidad de consulta y carga.
- Utilizar mecanismos de rollback: Configurar procedimientos para revertir cambios en caso de errores en la carga de datos.
- Registrar estadísticas de carga: Monitorear métricas clave como tiempo de ejecución, número de registros cargados y errores encontrados.
- Minimizar el impacto en la base de datos destino: Evitar bloqueos y degradaciones de rendimiento al distribuir las cargas en períodos de baja actividad.

6. MONITOREO Y CONTROL DE CALIDAD

El monitoreo de procesos ETL garantiza la detección temprana de errores y la calidad de los datos procesados.

6.1. Registro de Ejecuciones (Logging)

- Registrar cada ejecución con detalles como inicio, fin, duración y errores encontrados.
- Utilizar herramientas de monitoreo para la detección de fallos en tiempo real.

6.2. Validación y Control de Calidad

- Implementar verificaciones de integridad de datos antes y después de cada proceso ETL.
- Comparar volúmenes de datos extraídos vs. cargados para detectar inconsistencias.

7. GESTIÓN DE VERSIONAMIENTO

El versionamiento de procesos ETL es fundamental para el control de cambios y la trazabilidad de modificaciones.

7.1. Uso de Git

- Mantener scripts ETL en repositorios Git con ramas organizadas, Dando cumplimiento a lineamiento de versionamiento de código de seguros ALFA.
- Escribir mensajes de commit claros y descriptivos.

7.2. Control de Versiones en Datos

Usar metadatos y tablas de auditoría para mantener el historial de cambios en los datos.

8. CONCLUSIÓN

El cumplimiento de estos estándares garantiza la confiabilidad y eficiencia de los procesos ETL en Seguros Alfa. Se recomienda la revisión periódica de estos lineamientos y su actualización conforme a las mejores prácticas de la industria.